Apache Hadoop Lab Setup & MapReduce Practical Manual

1-Click Windows Lab PC Setup • Fast Shortcuts vs. Step-by-Step 1-by-1 Manual Reference

Target Environment: College / University Lab Windows PCs (WSL2) & Linux	**Apache Hadoop Version:** 3.3.6 / 3.4.x (Stable)
Lab 1-Liner: irm https://raw.githubusercontent.com/.../setup-lab.ps1 | iex	**Desktop Placement:** Automatically saves to Desktop\Hadoop
Dual MapReduce Tracks: 1-Click Fast Shortcut & Raw 1-by-1 Viva Commands	**Cluster Supervisor:** ran-hadoop (Auto-shutdown on exit)

1. Computer Lab Fast Setup (1 to 3 Line Commands)

In university computer labs, setting up Java, WSL, SSH keys, and Hadoop configuration manually takes up the entire practical session. Use these 1-to-3 line automated commands to configure the lab PC in seconds:

Option A: Windows Lab PC (PowerShell 1-Liner - Recommended for Labs)

Open PowerShell on the Windows lab computer and run this single command:

```
irm https://raw.githubusercontent.com/fridayslifes/Hadoop/main/setup-lab.ps1 | iex
```

What this does: Downloads the repo to your **Windows Desktop (%USERPROFILE%\Desktop\Hadoop)**, detects/launches WSL, configures OpenJDK 11 and Hadoop, and creates Desktop shortcuts for 'Start Hadoop' and 'Run WordCount'.

Option B: Linux / WSL Terminal 1-Liner

If you are already inside a WSL or Linux terminal, run this one-liner:

```
curl -sSL https://raw.githubusercontent.com/fridayslifes/Hadoop/main/install.sh | bash
```

Option C: Git Clone 2-Liner (if Git is available)

```
git clone https://github.com/fridayslifes/Hadoop.git ~/Desktop/Hadoop
cd ~/Desktop/Hadoop && ./install.sh
```

2. Starting & Stopping the Cluster ('ran-hadoop')

Standard Hadoop commands (start-dfs.sh) run background daemons that leave ports locked and consume gigabytes of RAM when you close your terminal. The **ran-hadoop** CLI solves this with automatic POSIX signal interception.

```
ran-hadoop
```

Cluster Lifecycle Controls:

- **Foreground Dashboard:** Keeps the cluster active and shows real-time daemon status.
- **Clean Stop:** Press **Ctrl + C** in the terminal to cleanly terminate all HDFS and YARN daemons.
- **Close Window Auto-Stop:** Closing the terminal tab sends a SIGHUP trap that safely cleans up all JVMs, leaving **zero orphaned processes**.

Hadoop Daemon	Process Role	Web Interface URL
NameNode	Master: Manages directory namespace and block map	http://localhost:9870

DataNode	Worker: Stores actual file data blocks on disk	http://localhost:9864
SecondaryNameNode	Checkpoint: Merges fsimage and editlogs	http://localhost:9868
ResourceManager	Master: Allocates CPU and memory across cluster	http://localhost:8088
NodeManager	Worker: Monitors local containers and CPU/RAM	http://localhost:8042

3. Track A: Fast 1-Line Shortcut Commands (Speed Run)

When you need to test WordCount or MapReduce immediately without typing repetitive HDFS commands, use the automated shortcut tool:

Shortcut 1: Run WordCount on Default Dataset

```
./run-wordcount.sh
```

Result: Automatically creates HDFS directories, ingests sample_input.txt, executes the MapReduce job on YARN, and displays the top 25 word frequencies in a formatted table.

Shortcut 2: Run WordCount on Any Custom File

You can pass any text file on your computer to process it immediately:

```
./run-wordcount.sh /path/to/my_big_data.txt
```

Shortcut 3: Global System Command

Because the installer links commands into /usr/local/bin, you can run from anywhere:

```
run-wordcount ~/sample_input.txt
```

4. Track B: Step-by-Step 1-by-1 Manual Commands (Lab Exam / Viva)

CRITICAL FOR PRACTICAL EXAMS: During lab evaluations, viva voce, and university practical examinations, professors and examiners require students to enter each command manually one by one without shortcuts. Follow this exact sequence:

> **■ Lab Practical Rules & Prerequisites**
>
> Make sure Hadoop is running first (either via ran-hadoop in Terminal 1 or by manually running start-dfs.sh && start-yarn.sh). Open a new terminal tab to execute the steps below.

Step 1: Verify Running Cluster Daemons

Confirm NameNode, DataNode, ResourceManager, and NodeManager are alive:

```
jps
```

Step 2: Create Input Directory in HDFS

Hadoop MapReduce jobs require input directories inside the distributed filesystem (HDFS):

```
hdfs dfs -mkdir -p /my_lab/input
```

Step 3: Upload Local File to HDFS

Transfer your local dataset (e.g. sample_input.txt) into HDFS:

```
hdfs dfs -put -f sample_input.txt /my_lab/input/
```

Step 4: Verify Input File in HDFS

List the directory contents and view the file inside HDFS:

```
hdfs dfs -ls /my_lab/input
hdfs dfs -cat /my_lab/input/sample_input.txt | head -n 10
```

Step 5: Clean Previous Output Directory (if re-running)

Hadoop will refuse to run if the destination output directory already exists:

```
hdfs dfs -rm -r -f /my_lab/output
```

Step 6: Execute the MapReduce Jar Step-by-Step

Run the official Hadoop MapReduce examples JAR via YARN:

```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar \
wordcount /my_lab/input /my_lab/output
```

Step 7: Inspect MapReduce Output Files in HDFS

MapReduce creates a success flag and partition files (part-r-00000):

```
hdfs dfs -ls /my_lab/output
hdfs dfs -cat /my_lab/output/part-r-00000 | head -n 25
```

Step 8: Export Output from HDFS back to Local Machine

Download the final computed results to your local Linux / Windows folder:

```
hdfs dfs -get /my_lab/output/part-r-00000 ./lab_results.txt
```

5. Bonus: Python Hadoop Streaming MapReduce

Many advanced college practicals ask for custom Python Mapper and Reducer scripts instead of Java JARs. Hadoop Streaming allows any standard Unix executable/script to act as a MapReduce worker.

A. The Mapper Script (mapper.py)

```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
line = line.strip()
words = line.split()
for word in words:
print(f"{word}\t1")
```

B. The Reducer Script (reducer.py)

```
#!/usr/bin/env python3
import sys

current_word = None
current_count = 0

for line in sys.stdin:
line = line.strip()
word, count = line.split('\t', 1)
count = int(count)
if current_word == word:
current_count += count
else:
if current_word:
print(f"{current_word}\t{current_count}")
current_word = word
current_count = count

if current_word:
print(f"{current_word}\t{current_count}")
```

C. Running Python Streaming on Hadoop:

```
chmod +x mapper.py reducer.py
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
-input /my_lab/input \
-output /my_lab/python_output \
-mapper ./mapper.py \
-reducer ./reducer.py
```

6. Quick Lab Practical Command Cheat Sheet

Lab Task / Action	Terminal Command
Windows Lab PC 1-Click Setup	irm https://raw.githubusercontent.com/.../setup-lab.ps1 | iex
Linux / WSL 1-Click Setup	curl -sSL https://raw.githubusercontent.com/.../install.sh | bash
Start Hadoop Cluster (with auto-shutdown)	ran-hadoop
Graceful Stop	Press **Ctrl + C** in ran-hadoop terminal
Fast MapReduce WordCount Shortcut	./run-wordcount.sh [data.txt]
Create HDFS Directory	hdfs dfs -mkdir -p <hdfs-path>
Copy Local File to HDFS	hdfs dfs -put -f <local-file> <hdfs-path>
List HDFS Directory Contents	hdfs dfs -ls <hdfs-path>
Print HDFS File to Terminal	hdfs dfs -cat <hdfs-path>
Download File from HDFS to Local	hdfs dfs -get <hdfs-path> <local-dest>
Leave HDFS Safe Mode	hdfs dfsadmin -safemode leave
HDFS NameNode Browser UI	http://localhost:9870
YARN Compute Browser UI	http://localhost:8088